

LangGraph 状态、Checkpoint 与 Store

DOC_ID: DOC-LG-001

说明：本文档为 RAGLab 项目生成的教学用合成语料，用于分块、检索、重排、引用和评测实验。它不是 LangChain/LangGraph 官方文档，也不应被当作 API 版本说明。文中的稳定标识（DOC_ID、SECTION_ID）用于构建可重复的检索评测集。

1. 图状态的作用

SECTION_ID: LG-STATE-01

LangGraph 的状态用于在节点之间传递业务数据和控制信息。状态不应只包含消息历史，还可以包含查询、检索结果、重试次数、审批状态和最终输出。

```
class RAGState(TypedDict):
    question: str
    rewritten_query: str
    documents: list
    retrieval_attempts: int
    answer: str
    grounded: bool
```

显式状态使每个节点的输入输出更容易测试，也便于恢复中断任务。

2. Checkpointer

SECTION_ID: LG-CP-01

Checkpoint 保存某个线程在图执行过程中的状态快照。它通常与 `thread_id` 配合，使系统能够在后续调用中恢复同一线程的执行上下文。

典型用途包括：

- 多轮对话状态持久化；
- Human-in-the-Loop 中断与恢复；
- 长任务失败后的断点续跑；
- 查询历史状态以进行调试。

Checkpoint 关注的是图运行状态和执行进度。

3. Store

SECTION_ID: LG-STORE-01

Store 更适合保存跨线程、跨会话可复用的长期信息，例如用户偏好、账户配置、长期记忆或共享实体资料。

Checkpointner 与 Store 的核心区别可以概括为：

组件	主要范围	典型内容
Checkpointner	单个线程的执行状态	messages、当前节点、审批状态、重试次数
Store	跨线程长期数据	用户偏好、长期记忆、共享知识

它们可以同时使用，但不应把所有业务数据库数据都塞入任何一个组件。

4. thread_id

SECTION_ID: LG-THREAD-01

thread_id 用于标识需要持续恢复的同一运行线程。不同测试样本必须使用不同 thread_id，否则前一条样本的消息和状态可能污染后一条样本。

在 Web 服务中，thread_id 应与用户会话或任务记录建立明确映射，而不是由前端随意覆盖。

5. interrupt 与恢复

SECTION_ID: LG-HITL-01

interrupt 用于在节点中暂停执行，并向外部返回需要人工处理的信息。人工提交结果后，系统使用恢复命令继续原图，而不是重新从入口执行所有节点。

例如 RAG 系统在执行高成本外部搜索前，可以暂停并请求用户确认；内容发布 Agent 在写入数据库前，也可以展示草稿并等待批准。

6. Reducer 与状态合并

SECTION_ID: LG-REDUCE-01

当多个节点都更新同一状态字段时，需要明确覆盖还是追加。消息列表通常使用追加语义，而 answer、current_step 等字段通常覆盖旧值。

不明确的合并规则会导致重复消息、旧状态残留或并行节点互相覆盖。

7. RAG 中使用 LangGraph 的边界

SECTION_ID: LG-RAG-01

简单的 2-Step RAG 不一定需要图。只有出现条件分支、循环、人工中断或多阶段验证时，LangGraph 才能体现优势，例如：

查询改写 -> 检索 -> 充分性判断

└ 足够 -> 生成 -> 忠实性检查

└ 不足 -> 再次改写 -> 检索

为了展示框架而把线性流程强行拆成很多节点，会增加维护成本。